CLIENT DISCOVERY BRIEF & SYSTEM REQUIREMENTS

Smart Document & Invoice Parsing Engine

Document Ref: SIL-BRF-2026-0812
Prepared By: Abdul (CEO, Silia Tech)
Client Name: Apex Logistics Ltd.
Date: August 12, 2026
Project Lead: Abdul
Status: Approved for Development

1. Client Overview & Background

Apex Logistics Ltd. is a mid-sized regional freight and logistics coordinator operating across four fulfillment hubs. They handle middle-mile transportation, warehousing, and inventory distribution for commercial retailers.

To keep supply chains moving efficiently, Apex partners with over 40 distinct sub-contractors, fuel vendors, maintenance providers, and toll operators.

2. Problem Statement & Business Operational Pain

Apex's accounts payable team currently receives between **150 to 250 vendor invoices daily**, primarily as unstructured PDF attachments sent via email.

Key Operational Bottlenecks Identified:

- **High Manual Overhead:** A team of three accounts payable specialists spends ~15 hours per week manually opening PDFs, re-keying invoice numbers, line-item breakdowns, tax subtotals, and final balances into their legacy ERP system.
- **Human Error & Late Fees:** Manual keying errors (e.g., transposed numbers or misread decimal points) lead to payment processing delays, duplicate disbursements, and occasional vendor late-payment surcharges.
- **Lack of Real-Time Visibility:** Because invoices sit in email inboxes for days before manual entry, management lacks visibility into daily operational expenditures and cash outflow forecasts.

3. Engagement Goals & Technical Objectives

Silia Tech has been engaged to design, build, and deploy an automated **Smart Document & Invoice Parsing Engine** to eliminate manual data entry.

Target Metric	Objective Baseline
Processing Time Reduction	Reduce manual data entry time by >85% across all hubs.
Data Extraction Accuracy	Maintain >95% accuracy on core financial metrics (Totals, Taxes, Vendor Names).
Turnaround Time	Process and validate single invoices in < 5 seconds per file.

4. Client Preferences & Constraints

During the technical discovery session, Apex Logistics outlined the following core preferences:

- **Zero Custom Training Datasets:** The system must handle invoices from *new or unseen vendors* without requiring manual template setup or training machine learning models for each layout.
- **Strict Type Enforcement:** Financial records must strictly adhere to database type formats (e.g., currency as floats/decimals, dates as YYYY-MM-DD). The system must fail gracefully rather than outputting malformed JSON data.
- **Human-in-the-Loop Verification:** Operational staff must be able to visually inspect extracted data side-by-side with the uploaded PDF before committing data or exporting JSON.
- **Lightweight Web UI:** Clean, browser-based interface requiring zero local software installation on staff workstations.

5. System Requirements & Specifications

Requirement ID	Categor y	Description & Specification
FR -01	File Ingestion	Support drag-and-drop upload of digital PDF documents (single and multi-page invoices).
FR -02	Text Extraction	Extract vendor details, invoice metadata, itemized billings, subtotal, tax, and total due.
FR -03	Structured Output	Enforce structured output via Pydantic schemas for 100% field type compliance.
FR -04	Data Export	Allow staff to inspect side-by-side and export validated records to JSON or tabular view.
NFR-01	Schema Validation	Utilize Pydantic strict typing to guarantee structure integrity on all model responses.
NFR-02	Maintainabilit y	Modular Python architecture separating extraction logic, AI models, and frontend UI.

6. Proposed Architecture & Stack Selection

Component	Selected Tech	Technical Rationale
User Interface	Streamlit	Rapid web deployment, native support for side-by-side preview, dynamic dataframes, and JSON viewing.
PDF Engine	pypdf	Fast, lightweight text extraction engine for digital PDF documents.
LLM Core	Google Gemini API (2.5 Flash)	High execution speed, minimal latency, and native support for structured JSON outputs.
Schema Enforcement	Pydantic v2	Guarantees type safety and validates nested line-item objects at runtime.

7. Delivery Roadmap

1. **Phase 1: Environment & Core Engine Setup (Days 1–2):** Set up virtual environment, configure Pydantic schemas, and integrate LLM API.
2. **Phase 2: Parsing & Extraction Pipeline (Days 3–4):** Build pypdf text extraction module and construct structured LLM prompt chain.
3. **Phase 3: Web Dashboard Development (Days 5–6):** Develop Streamlit UI with side-by-side document viewer, table preview, and export controls.
4. **Phase 4: Acceptance Testing & Delivery (Day 7):** Execute test suite using Apex sample invoices and hand over repository.
